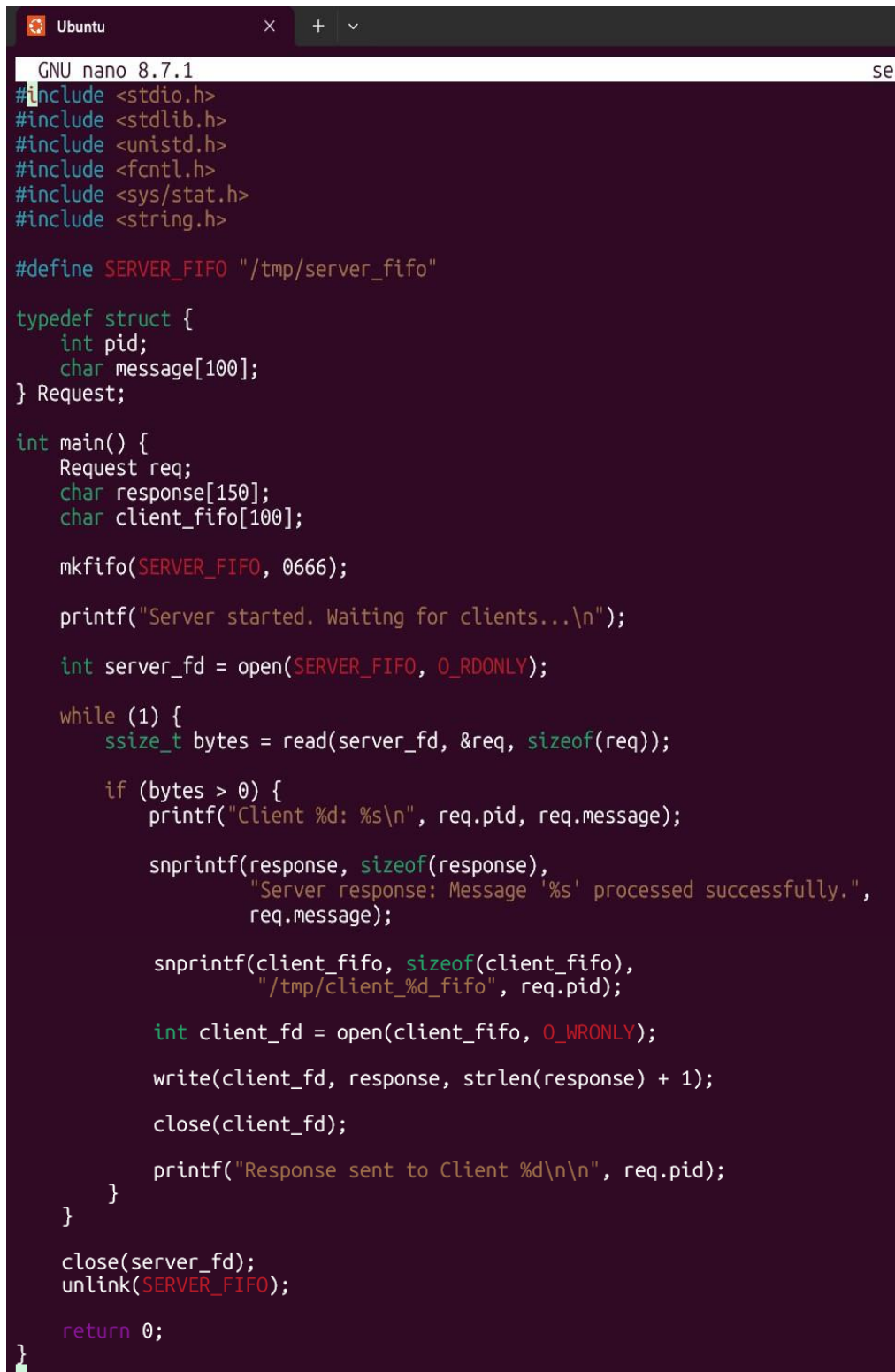


Practical Week -6

Task 1:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.



```
GNU nano 8.7.1 ser
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define SERVER_FIFO "/tmp/server_fifo"

typedef struct {
    int pid;
    char message[100];
} Request;

int main() {
    Request req;
    char response[150];
    char client_fifo[100];

    mkfifo(SERVER_FIFO, 0666);

    printf("Server started. Waiting for clients...\n");

    int server_fd = open(SERVER_FIFO, O_RDONLY);

    while (1) {
        ssize_t bytes = read(server_fd, &req, sizeof(req));

        if (bytes > 0) {
            printf("Client %d: %s\n", req.pid, req.message);

            snprintf(response, sizeof(response),
                "Server response: Message '%s' processed successfully.",
                req.message);

            snprintf(client_fifo, sizeof(client_fifo),
                "/tmp/client_%d_fifo", req.pid);

            int client_fd = open(client_fifo, O_WRONLY);

            write(client_fd, response, strlen(response) + 1);

            close(client_fd);

            printf("Response sent to Client %d\n\n", req.pid);
        }
    }

    close(server_fd);
    unlink(SERVER_FIFO);

    return 0;
}
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define SERVER_FIFO "/tmp/server_fifo"

typedef struct {
    int pid;
    char message[100];
} Request;

int main() {
    Request req;
    char client_fifo[100];
    char response[150];

    req.pid = getpid();

    printf("Enter message: ");
    fgets(req.message, sizeof(req.message), stdin);

    req.message[strcspn(req.message, "\n")] = '\0';

    snprintf(client_fifo, sizeof(client_fifo),
             "/tmp/client_%d_fifo", req.pid);

    mkfifo(client_fifo, 0666);

    int server_fd = open(SERVER_FIFO, O_WRONLY);
    write(server_fd, &req, sizeof(req));
    close(server_fd);

    int client_fd = open(client_fifo, O_RDONLY);
    read(client_fd, response, sizeof(response));

    printf("%s\n", response);

    close(client_fd);

    unlink(client_fifo);

    return 0;
}
```

```

nocturnal@nocturnal:~$ nano server.c
nocturnal@nocturnal:~$ gcc server.c -o server
server.c: In function 'main':
server.c:33:76: warning: 'snprintf' output may be truncated before the last format character [-Wformat-truncation=]
   33 |         "Server response: Message '%s' processed successfully.",
      |                                     ^
server.c:32:13: note: 'snprintf' output between 52 and 151 bytes into a destination of size 150
   32 |         snprintf(response, sizeof(response),
      |         ^~~~~~
   33 |             "Server response: Message '%s' processed successfully.",
      |             ~~~~~~
   34 |             req.message);
      |             ~~~~~~

```

Output:

```

nocturnal@nocturnal:~$ nano client.c
nocturnal@nocturnal:~$ gcc client.c -o client
nocturnal@nocturnal:~$ ./client
Enter message: Hi
Server response: Message 'Hi ' processed successfully.
nocturnal@nocturnal:~$

```

```

nocturnal@nocturnal:~$ ./server
Server started. Waiting for clients...
Client 1338: Hi
Response sent to Client 1338

```

Task-2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.



The image shows a terminal window with two tabs, both labeled 'Ubuntu'. The active tab displays the GNU nano 8.7.1 text editor. The code is a C program for signal handling. It includes headers for stdio, stdlib, signal, and unistd. A signal_handler function is defined to handle SIGINT, SIGTERM, and SIGUSR1 signals, printing their names when received. The main function sets up these signal handlers, prints the process ID, and enters a loop where it prints 'Program is running...' and sleeps for 3 seconds. The program returns 0 at the end.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal) {
    if (signal == SIGINT)
        printf("\nSIGINT received.\n");
    else if (signal == SIGTERM)
        printf("\nSIGTERM received.\n");
    else if (signal == SIGUSR1)
        printf("\nSIGUSR1 received.\n");
}

int main() {
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

Output:

```
nocturnal@nocturnal:~$ kill -SIGINT 1402
nocturnal@nocturnal:~$ kill -SIGUSR1 1200
-bash: kill: (1200) - No such process
nocturnal@nocturnal:~$ kill -SIGUSR1 1402
nocturnal@nocturnal:~$ kill -SIGTERM 1402
nocturnal@nocturnal:~$
```

```
nocturnal@nocturnal:~$ nano signals.c
nocturnal@nocturnal:~$ gcc signals.c -o signals
nocturnal@nocturnal:~$ ./signals
Process ID: 1402
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGINT received.
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGUSR1 received.
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGTERM received.
Program is running...
Program is running...
Program is running...
```